

# ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS

## DEPARTMENT OF INFORMATICS

### ATHANASIOS GOURDOMICHALIS

#### Operating Systems

## 1. Introduction

**i** When two or more processes or threads access the same resource or write to the same memory locations, synchronization is required to prevent **race conditions**. For synchronizing processes and threads, the following tools are provided:

- **Processes:** Semaphores
- **Threads:** Mutexes and Condition Variables

In this assignment, you will become familiar with process synchronization across different scenarios. To complete the assignment, download the relevant code template from eclass, named **code2026.tar**. Extract the tar archive using the command **tar xvf code2026.tar**, which will create the directory code2026.

- To compile the code, simply run **make** inside the directory generated by extracting the tar file.

## 2. Process Synchronization

**i** In this task, you are required to implement the programs **multiproc1** and **multiproc2**.

**multiproc1** takes one or more command-line arguments. The first argument (mandatory) is an integer number. Let us name it **rep** (short for repetitions). The remaining arguments are random strings. When executed, **multiproc1** must create as many child processes as there are input strings, and each process must print its corresponding string **rep** times. For printing, instead of using **printf()**, you must use **display(char \*)**, which is provided in **util.c**.

When we write the initial code and run it for the first time with two or more child processes, you will notice that the outputs of the different

processes are interleaved character by character. In this exercise, we must use synchronization mechanisms to guarantee **mutual exclusion** during printing-allowing only one process to print at a time. This ensures that while output lines may appear in a non-deterministic order, each line remains "clean" without its characters being mixed with characters from other lines.

**Attention:** You must not modify `display()` in any way, nor are you allowed to use `sleep()` or `usleep()`. Furthermore, it is incorrect to simply execute one process first, then the second, and so on sequentially. The processes must run concurrently in true parallel fashion.

**i** For example, a valid execution of `multiproc1` could look like this:

```
$/multiproc1 3 Informatics Department
[1000,123456789] Informatics
[1000,123456789] Informatics
[1001,987654321] Department
[1000,123456789] Informatics
[1001,987654321] Department
[1001,987654321] Department
```

where one child process (with PID 1000) prints "Informatics" 3 times, and the second child process (with PID 1001) prints "Department" 3 times. To help you better understand what is happening, `display()` also prints both the process's PID and the thread's TID.

The `multiproc2` program must do the exact same thing as the previous one, with the additional requirement that each child process must call the `init()` function (provided in `util.c`) once before calling `display()`.

Most importantly, **no child process may begin invoking `display()` before all processes have completed their call to `init()`!**

